# Lecture 40 — SVD and capstone launch

**Fri Dec 4**

Last updated: October 8, 2026.

**Textbook:** Boyd & Vandenberghe, *VMLS* (PDF)

**Strang / MIT (optional):** Reading: Strang/MIT SVD slides (PDF)
Video: 18.06 Lec 29 — singular value decomposition · Short SVD overview (Strang) · MIT 18.06 playlist (Strang)

Reading: selected slides from the Strang/MIT SVD slides (not all 17 pages), capstone launch, Lachniet Ch. 5 (SVD) and Ch. 7 (image compression). Related objectives: Lectures. Schedule.

Bring paper and a laptop. Warm-up, fill-in notes, board examples, and **Your turn**. See the schedule for today's quiz, lab, and capstone launch.

## Learning objectives

By the end of class, you should be able to:

1. Interpret $A = U\Sigma V^{\mathsf{T}}$ conceptually.
2. Construct a rank-$k$ approximation computationally.
3. Compare singular-value decay for an image/frame and a frame-difference matrix.
4. Encode three frames either independently or as $F_1$, $D_2$, $D_3$.
5. Compare reconstruction error and storage for the two approaches.
6. Formulate a capstone question involving accuracy, storage, runtime, and resource use.

*(Objectives 3–6 are launched today and carried in the capstone packet.)*

## Warm-up (after quiz)

A grayscale image (or one video frame) is an $m \times n$ matrix of pixel intensities.

(a) Roughly how many numbers store the full matrix?

(b) If most "information" lived in a few patterns, why might a factored low-rank form save storage?

## Fill in

**SVD (conceptual).** Every real $m \times n$ matrix factors as

$$A = U\Sigma V^{\mathsf{T}},$$

where the columns of $U$ and $V$ are orthonormal (left / right **singular vectors**) and $\Sigma$ is rectangular "diagonal" with nonnegative **singular values**

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$$

on its main diagonal. Think: $A$ = rotate/reflect $\rightarrow$ stretch by $\sigma_i$ $\rightarrow$ rotate/reflect.

**Rank-$k$ approximation.** Keep the largest $k$ singular values (and matching singular-vector columns); set the rest to zero. The truncated product

$$A_k = U_k \Sigma_k V_k^{\mathsf{T}}$$

is the best rank-$k$ approximation in the usual 2-norm / Frobenius sense (details in the slides / capstone).

**Software sketch** (capstone / later):

| Task | Octave | Python |
|---|---|---|
| SVD | `[U,S,V] = svd(A)` | `U, S, Vt = np.linalg.svd(A, full_matrices=False)` |
| Rank-$k$ rebuild | use first $k$ columns / $\sigma$'s | `U[:,:k] @ np.diag(S[:k]) @ Vt[:k,:]` |

**Capstone launch (one question).** For a short clip of frames: when is it better to compress each frame alone versus encode $F_1$ and differences $D_2 = F_2 - F_1$, $D_3 = F_3 - F_2$? Trade off **accuracy**, **storage**, **runtime**, and **resource use** — including the cost of computing the SVD itself. Full prompts live in the capstone packet; studio is Mon Dec 7.

## Worked in class (short)

**W1.** Read $\Sigma$. Suppose a $4 \times 3$ matrix has singular values $\sigma = (10, 2, 0.1)$. What is rank$(A)$ (numerically "full" vs tiny)? For a rank-1 approximation, which $\sigma$ do you keep?

**W2.** Storage cartoon. Full $m \times n$ needs $mn$ entries. A rank-$k$ factorization stores about $k(m + n + 1)$ numbers (columns of $U_k$, $V_k$, and $k$ singular values). When is $k(m + n + 1) \ll mn$?

**W3.** Frames vs differences (launch). Three frames $F_1, F_2, F_3$. Plan A: compress each. Plan B: compress $F_1$, $D_2 = F_2 - F_1$, $D_3 = F_3 - F_2$. In one sentence each: when might Plan B win on storage? What must you still check (error / runtime)?

## Your turn

**Y1.** In $A = U\Sigma V^{\mathsf{T}}$, what do the singular values measure conceptually (one phrase)?

**Y2.** True or false: computing an SVD is always "free," so low-rank multiplication is always faster overall.

**Y3.** Write one capstone question you care about (accuracy, storage, runtime, or resource use) for frames vs frame-differences. One sentence is enough — expand in studio.

## Answers (Your turn)

Y1. Stretch factors / importance of each singular-vector mode (how strongly $A$ amplifies that direction).

Y2. **False.** The factorization itself can be expensive; low-rank reuse only pays off when approximation quality is good **and** you reuse the factors enough (see capstone notes).

Y3. Any clear tradeoff question is fine — e.g. "For my three frames, does difference encoding cut storage at the same relative error, after counting SVD time?"